DAY 34 | HANDS-ON ACTIVITIES

LLM Deployment and Lifecycle: Problem Statements

Seven problems building a production-style LLM service. The model is simulated, so everything runs without a key or GPU.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Pick an Architecture NEW (v4)

Level: Warm-up | **Concepts:** hosting options, managed runtimes

Recommend an architecture for: a bank that cannot send data outside its tenant; a start-up prototype; a call centre with 50 million requests a month; a team using three model providers; a two-week agent pilot with no platform team.

Sample run / expected result

```
Bank: model hosted in its own cloud tenant
Start-up: hosted API
Call centre: evaluate a fine-tuned small model, self-hosted
Three providers: AI gateway
Agent pilot: managed agent runtime, retention checked
```

Hints

- Ask about data rules, volume, skills, budget and retention.

Problem 2: Fail-fast Settings

Level: Easy | **Concepts:** environments, secrets

Write settings loading for dev, test and prod. In prod, missing secrets must stop the service at startup with a clear message. Never print a secret value.

Sample run / expected result

```
APP_ENV=prod without LLM_API_KEY -> RuntimeError: LLM_API_KEY missing in prod
```

Hints

- Print 'set' or 'not set', never the value.

Problem 3: Version and Roll Back

Level: Medium | **Concepts:** prompt, dataset and embedding versioning

Keep a release registry with two manifests (prompt, golden set and embedding settings). Promote 1.9.0 to prod, detect that its embedding model differs, then roll back to 1.8.0 in one call.

Sample run / expected result

```
prod -> 1.9.0 (embed: embed-v4, index must be rebuilt)
rollback -> prod = 1.8.0
```

Hints

- Changing the embedding model means rebuilding the whole index.

Problem 4: AI Gateway with Budgets NEW (v4)

Level: Medium | **Concepts:** gateway, allow-list, budgets

Build a FastAPI gateway that checks a client key, enforces a model allow-list and a daily token budget per team, and logs every call. Test allowed, wrong model, over budget and bad key.

Sample run / expected result

```
200 ok | 403 model not allowed | 429 budget exceeded | 401 bad key
```

Hints

- Store hashes of client keys, not the keys.
- Prices and budgets are placeholders.

Problem 5: Short-lived Tokens for a Stateless MCP Server NEW (v4)

Level: Challenging | **Concepts:** stateless MCP, scoped tokens, no long-lived keys

Issue HMAC-signed tokens with scopes and a 5-minute expiry. Protect a stateless /mcp endpoint with tools/list and tools/call. Show that a read-only token cannot call the refund tool and an expired token fails.

Sample run / expected result

```
tools/list -> 2 tools
call lookup_ticket (scope tickets.read) -> 200
call refund (needs refunds.write) -> 403
expired token -> 401
```

Hints

- Put exp and scopes inside the signed payload.
- This is a simplified protocol; Day 28 uses the real MCP SDK.

Problem 6: Concurrency Limit and Streaming

Level: Medium | **Concepts:** asyncio, semaphore, StreamingResponse

Simulate a provider that rejects more than 20 calls in flight. Send 100 requests unlimited and with a semaphore of 10. Then measure time to first chunk vs total time for the streaming endpoint.

Sample run / expected result

```
unlimited: 80 rejected | semaphore 10: 0 rejected
streaming: first chunk about 100 ms, full answer about 320 ms
```

Hints

- TestClient buffers streams; measure with httpx.stream against a running uvicorn server.

Problem 7: Canary and Rollback Rules

Level: Stretch | **Concepts:** canary, rollback criteria

Route 10% of users to release 1.9 by a stable hash of their user id. Given canary and baseline metrics, decide promote, hold or rollback, and write rollback test notes.

Sample run / expected result

```
canary share: 9.3% of 5,000 users (close to 10%)
error rate 3.1% vs 1.0% -> ROLLBACK
```

Hints

- Use hashlib, not Python's hash(), which changes between runs.